

Fundación Fulgor

Diseño Digital Avanzado

GP01

Nombre	DNI
Corvalán, Abel Nicolás	41.220.050

Docentes	Ing. Pola, Ariel Cayuela, Pablo.
----------	-------------------------------------

Córdoba, República Argentina
30 de septiembre de 2026

Índice

1. Parte A — Propiedades del código	2
1.1. Matrices y parámetros	2
1.2. Dimensión y tasa	2
1.3. Overhead	2
1.4. Codificación de información de prueba	3
1.5. Distancia mínima y capacidad de corrección	3
1.5.1. Distancia mínima	3
1.5.2. Capacidad de detección	3
1.6. Detalles del código	3
1.6.1. Miscorrection	3
1.7. Decodificación manual	4
1.8. Palabra recibida 1: <i>0xA5D9A6</i>	4
1.8.1. Caso 1	4
1.8.2. Caso 2	5
1.9. Palabra recibida 2: <i>0xA5F9A4</i>	5
1.9.1. Caso 1	5
1.9.2. Caso 2	5
1.9.3. Caso 3	5
1.9.4. Caso 4	6
1.10. Palabra recibida 3: <i>0xA5C9AA</i>	6
1.10.1. Caso 1	6
1.10.2. Caso 2	7
1.10.3. Caso 3	7
1.10.4. Caso 4	7
2. Parte B — Modelo de referencia en software	8
2.1. Codificador y decodificador de referencia	8
2.2. Caracterización del decodificador	8
2.3. Clase GaloisField en GF(2)	9
2.4. Codificador y decodificador de referencia	10
2.5. Caracterización del decodificador	10
3. Parte C — Implementación RTL	12
3.1. Desarrollo y verificación de módulos comunes	12
3.2. Analisis: Posible modificacion del decoder	12
3.3. Implementación RTL del encoder	12
3.4. Resumen del encoder	12
3.5. Validación del encoder	15
3.6. Implementación RTL del decoder	15
3.7. Resumen del decoder	18
3.8. Validacion del decoder	18
3.9. Correcciones en el decoder	19
3.10. Integración del top level	20
3.11. Verificación del top level	22

1. Parte A — Propiedades del código

1.1. Matrices y parámetros

Se tiene en principio, la matriz $\mathbf{B}$ para luego armar las matrices $\mathbf{G}$ y $\mathbf{H}$.

A continuación se muestran las matrices $\mathbf{G}$ y $\mathbf{H}$.

$$\mathbf{G} = \left[\begin{array}{cccccccccccc|cccccccccccc} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{array} \right]$$

$$\mathbf{H} = \left[\begin{array}{cccccccccccc|cccccccccccc} 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{array} \right]$$

En el archivo reference.ods", se verifican las operaciones $\mathbf{G} \cdot \mathbf{H}^T = 0$ y $\mathbf{B}^2 = \mathbf{I}$

1.2. Dimensión y tasa

Respecto a la dimensión tenemos que la matriz $\mathbf{G}$ pertenece al campo de Galois $GF(2)^{12 \times 24}$. De esta forma tenemos $k = 12$ bits de información y $n = 24$ bits totales, incluyendo redundancia. El código Golay en este caso resulta $(n, k, d_{min}) = (24, 12, 8)$.

La tasa se calcula como el cociente entre la cantidad de bits de información y los bits totales.

$$R = \frac{k}{n} = \frac{12}{24}$$

$$R = 0,5$$

Es decir que se tiene la misma cantidad de bits de información como de redundancia.

1.3. Overhead

La cantidad de bits adiciones que se agregan para poder realizar la codificación en relación con la información original es la siguiente. Se tienen los bits de información (k) y los bits en la palabra codificada (n). Para el código de Golay, en este caso se tiene:

$$overhead = n - k = 24 \text{ bits} - 12 \text{ bits}$$

$$overhead = 12 \text{ bits}$$

1.4. Codificación de información de prueba

Se hace la codificación de la siguiente palabra:

$$m = 0xA5C$$

La palabra codificada resulta en:

$$v = 0xA5C9A5$$

Luego multiplicando la palabra codificada v por $\mathbf{H}^T$ se tiene:

$$v \cdot \mathbf{H}^T = 0$$

Todo el procedimiento de cálculo manual se muestra en el archivo *reference.ods*.

1.5. Distancia mínima y capacidad de corrección

Se tiene la distribución de pesos para el total de las palabras del código. La tabla se muestra en la siguiente figura:

Peso	0	8	12	16	24
Cantidad de palabras	1	759	2576	759	1

Figura 1: Tabla de distribución de pesos

1.5.1. Distancia mínima

El código Golay es un código lineal, por lo que la distancia mínima es igual al menor peso no nulo de una palabra del código. Los pesos no nulos son 8, 12, 16 y 24. Por lo tanto se tiene que la distancia mínima es:

$$d_{min} = 8$$

1.5.2. Capacidad de detección

Respecto a la capacidad de detección se tiene que el cálculo para el mismo es el siguiente:

$$d_{min} - 1 = 8 - 1 = 7$$

$$capacidad\ de\ deteccin = 7$$

El código Golay puede detectar hasta 7 errores.

1.6. Detalles del código

1.6.1. Miscorrection

El algoritmo decodificador está diseñado para corregir patrones de hasta 3 errores. A raíz de esto puede ocurrir que se transmita una codeword c_1 . La misma llega al decodificador con 5 errores. Entonces se tiene que la palabra recibida es:

$$r_1 = c_1 \oplus e_5$$

Puede existir otra codeword c_2 a distancia $d(c_1, c_2) = 8$ de c_1 . El error introducido puede acortar esta distancia de hamming entre la palabra recibida r y la codeword c_2 . Obteniendo en el caso más desfavorable resultante con $d(r, c_2) = 3$. El decodificador en estas condiciones corrige 3 errores, pero no se corrige considerando que se quiso recibir r . Se obtiene a la salida del decodificador una codeword errónea ($c_2 \neq c_1$). En el proceso entonces, se recibe una palabra con errores que excede la capacidad de corrección del decodificador. Se encuentra un patrón de error que parece válido. Se aplica la corrección y se obtiene otra codeword válida, pero no la que corresponde a la palabra que en realidad se quiso recibir.

1.7. Decodificación manual

Se codifican palabras recibidas propuestas siguiendo el algoritmo de cuatro casos. Se hace el desarrollo en hoja cálculo adjunta en repositorio. En la siguiente figura se presentan los casos de obtención de la máscara de error:

Caso	Condición	Patrón de error
1	$w(\mathbf{s}) \leq 3$	$\mathbf{e} = (\mathbf{0} \mid \mathbf{s})$
2	$\exists i : w(\mathbf{s} \oplus \mathbf{b}_i) \leq 2$	$\mathbf{e} = (\mathbf{u}_i \mid \mathbf{s} \oplus \mathbf{b}_i)$
3	$w(\mathbf{q}) \leq 3$, con $\mathbf{q} = \mathbf{s} \cdot \mathbf{B}$	$\mathbf{e} = (\mathbf{q} \mid \mathbf{0})$
4	$\exists i : w(\mathbf{q} \oplus \mathbf{b}_i) \leq 2$	$\mathbf{e} = (\mathbf{q} \oplus \mathbf{b}_i \mid \mathbf{u}_i)$
5	ninguna de las anteriores	$\mathbf{o_uncorrectable}$

Figura 2: Casos de evaluación para corrección

Las palabras recibidas propuesta para evaluar en el codificador son las siguientes:

r_1	r_2	r_3
0xA5D9A6	0xA5F9A4	0xA5C9AA

Se calculan los vectores s y q . Estos vectores se obtienen realizando las siguientes operaciones:

$$s = r[23 : 12] \cdot \mathbf{B} \oplus r[11 : 0]$$

$$q = s \cdot \mathbf{B} = (r[23 : 12] \cdot \mathbf{B} \oplus r[11 : 0]) \cdot \mathbf{B}$$

$$q = r[23 : 12] \oplus (r[11 : 0] \cdot \mathbf{B})$$

A continuación se muestra la matriz $\mathbf{B}$:

$$\mathbf{B} = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Se obtienen los siguientes conjuntos de vectores s y q para las palabras a decodificar:

r	s	q
0xA5D9A6	0xEAA	0xEAA
0xA5F9A4	0x1B2	0xEAA
0xA5C9AA	0x00F	0x7BC

1.8. Palabra recibida 1: 0xA5D9A6

1.8.1. Caso 1

Luego se evalúa el peso del vector s y la condición correspondiente al primer caso:

$$weight(s) \leq 3 = 7 \leq 3 = FALSE$$

Por lo que no se cumple con la primera condición.

1.8.2. Caso 2

Se evalúa si algún peso de las operaciones $s \oplus b_i$ es menor a 2. Esto es:

$$weight(S \oplus b_i) \leq 2$$

Expresión	Peso
$weight(S \oplus b_0)$	6
$weight(S \oplus b_1)$	6
$weight(S \oplus b_2)$	10
$weight(S \oplus b_3)$	4
$weight(S \oplus b_4)$	8
$weight(S \oplus b_5)$	6
$weight(S \oplus b_6)$	8
$weight(S \oplus b_7)$	4
$weight(S \oplus b_8)$	6
$weight(S \oplus b_9)$	6
$weight(S \oplus b_{10})$	4
$weight(S \oplus b_{11})$	2

Entonces se tiene que la operación que cumple con la condición es:

$$weight(S \oplus b_{11}) = 2 \leq 2$$

Entonces la máscara de error se construye de la siguiente forma:

$$\mathbf{e} = (u_i | (s \oplus b_i))$$

En este caso resulta:

$$\boxed{\mathbf{e} = 0x000001}$$

1.9. Palabra recibida 2: 0xA5F9A4

1.9.1. Caso 1

Luego se evalúa el peso del vector s y la condición correspondiente al primer caso:

$$weight(s) \leq 3 = 5 \leq 3 = FALSE$$

Por lo que no se cumple con la primera condición.

1.9.2. Caso 2

Se evalúa si algún peso de las operaciones $s \oplus b_i$ es menor a 2. Esto es:

$$weight(S \oplus b_i) \leq 2$$

Ninguna de las operaciones cumple con la condición.

1.9.3. Caso 3

Se calcula el peso del vector q . Se tiene:

$$weight(q) = 7$$

$$weight(q) \leq 3 = 7 \leq 3 = FALSE$$

Por lo que no se cumple la condición.

Expresión	Peso
$\text{weight}(S \oplus b_0)$	6
$\text{weight}(S \oplus b_1)$	6
$\text{weight}(S \oplus b_2)$	6
$\text{weight}(S \oplus b_3)$	4
$\text{weight}(S \oplus b_4)$	6
$\text{weight}(S \oplus b_5)$	8
$\text{weight}(S \oplus b_6)$	6
$\text{weight}(S \oplus b_7)$	4
$\text{weight}(S \oplus b_8)$	6
$\text{weight}(S \oplus b_9)$	8
$\text{weight}(S \oplus b_{10})$	6
$\text{weight}(S \oplus b_{11})$	8

1.9.4. Caso 4

Se evalúa si algún peso de las operaciones $q \oplus b_i$ es menor a 2. Esto es:

$$\text{weight}(q \oplus b_i) \leq 2$$

Expresión	Peso
$\text{weight}(q \oplus b_0)$	6
$\text{weight}(q \oplus b_1)$	6
$\text{weight}(q \oplus b_2)$	10
$\text{weight}(q \oplus b_3)$	4
$\text{weight}(q \oplus b_4)$	8
$\text{weight}(q \oplus b_5)$	6
$\text{weight}(q \oplus b_6)$	8
$\text{weight}(q \oplus b_7)$	4
$\text{weight}(q \oplus b_8)$	6
$\text{weight}(q \oplus b_9)$	6
$\text{weight}(q \oplus b_{10})$	4
$\text{weight}(q \oplus b_{11})$	2

Entonces se tiene que la operación que cumple con la condición es:

$$\text{weight}(q \oplus b_{11}) = 2 \leq 2$$

Entonces la máscara de error se construye de la siguiente forma:

$$\mathbf{e} = ((q \oplus b_i)|u_i)$$

En este caso resulta:

$$\boxed{\mathbf{e} = 0x003001}$$

1.10. Palabra recibida 3: 0xA5C9AA

1.10.1. Caso 1

Se evalúa el peso del vector s y la condición correspondiente al primer caso:

$$\text{weight}(s) \leq 3 = 4 \leq 3 = FALSE$$

Por lo que no se cumple con la primera condición.

Expresión	Peso
$\text{weight}(S \oplus b_0)$	3
$\text{weight}(S \oplus b_1)$	5
$\text{weight}(S \oplus b_2)$	5
$\text{weight}(S \oplus b_3)$	9
$\text{weight}(S \oplus b_4)$	9
$\text{weight}(S \oplus b_5)$	7
$\text{weight}(S \oplus b_6)$	5
$\text{weight}(S \oplus b_7)$	5
$\text{weight}(S \oplus b_8)$	5
$\text{weight}(S \oplus b_9)$	9
$\text{weight}(S \oplus b_{10})$	7
$\text{weight}(S \oplus b_{11})$	7

1.10.2. Caso 2

Se evalúa si algún peso de las operaciones $s \oplus b_i$ es menor a 2. Esto es:

$$\text{weight}(S \oplus b_i) \leq 2$$

Ninguna de las operaciones cumple con la condición.

1.10.3. Caso 3

Se calcula el peso del vector q . Se tiene:

$$\text{weight}(q) = 8$$

$$\text{weight}(q) \leq 3 = 8 \leq 3 = \text{FALSE}$$

Por lo que no se cumple la condición.

1.10.4. Caso 4

Se evalúa si algún peso de las operaciones $q \oplus b_i$ es menor a 2. Esto es:

$$\text{weight}(q \oplus b_i) \leq 2$$

Expresión	Peso
$\text{weight}(q \oplus b_0)$	7
$\text{weight}(q \oplus b_1)$	7
$\text{weight}(q \oplus b_2)$	7
$\text{weight}(q \oplus b_3)$	7
$\text{weight}(q \oplus b_4)$	7
$\text{weight}(q \oplus b_5)$	3
$\text{weight}(q \oplus b_6)$	3
$\text{weight}(q \oplus b_7)$	3
$\text{weight}(q \oplus b_8)$	9
$\text{weight}(q \oplus b_9)$	5
$\text{weight}(q \oplus b_{10})$	5
$\text{weight}(q \oplus b_{11})$	5

Se tiene que ninguna de las operaciones cumple con la condición.

De esta forma hace que la palabra recibida sea clasificada como incorregible:

$$\mathbf{e} = 0x000000$$

2. Parte B — Modelo de referencia en software

2.1. Codificador y decodificador de referencia

Se implementa el codificador matricial y el decodificador en los archivos `GolayEncoder.py` y `GolayDecoder.py`, respectivamente.

La tabla de síndromes se encuentra en el archivo `fec-lab2/model/outputs/syndromes/error_syndromes.txt`. Estos son generados con los scripts del golden model. Los vectores se presentan de la siguiente forma:

1. Palabra recibida	2. Palabra incorregible
---------------------	-------------------------

Tabla 1: Descripción de las columnas de salida del decodificador.

La totalidad de los errores que se pueden producir dentro de los 24 bits de una palabra recibida en el decoder son generados de la siguiente forma:

El orden en que se encuentran las variables es el siguiente:

1. Palabra recibida	2. Mensaje	3. Máscara de error	4. Palabra corregida	5. Palabra incorregible
---------------------	------------	---------------------	----------------------	-------------------------

Tabla 2: Descripción de las columnas de salida del decodificador.

Los valores decodificados se encuentran en el archivo `../outputs/decoder/decoder_top_testing/codeword_with_e`.

2.2. Caracterización del decodificador

Para cada peso entre 0 y 4, se recorre la totalidad de los patrones de error correspondientes a dicho peso. Estos vectores son generados en el golden model `golay_decoder.ipynb`.

Luego, estos vectores son procesados por un objeto decodificador para obtener el archivo de vectores `outputs/decoder/decoder_top_testing/codeword_with_errors.txt`.

Este archivo tiene la siguiente convención de vectores.

1. Palabra recibida	2. Mensaje	3. Máscara de error	4. Palabra corregida	5. Palabra incorregible
---------------------	------------	---------------------	----------------------	-------------------------

El script con el cual se genera el conjunto de vectores es el siguiente:

Listing 1: Generación de los vectores de prueba del decodificador

```
1 def write_codewords_with_error(filename, codeword, max_errors):
2     n = len(codeword)
3     received_word_array_with_errors = []
4     # Convertir codeword [0,1,1,...] -> entero
5     codeword_int = int("".join(map(str, codeword)), 2)
6     # Todas las posiciones posibles donde colocar los errores, por peso
7     # C(24,0) + C(24,1) + C(24,2) + C(24,3) + C(24,4) = 12951
8     error_masks = [
9         sum(1 << (n - 1 - pos) for pos in error_positions)
10        for n_errors in range(max_errors + 1)
11        for error_positions in combinations(range(n), n_errors)
12    ]
13    with open(filename, "w") as file:
14        for mask in error_masks:
15            received_word = codeword_int ^ mask
16            received_bits = np.array(
17                [int(bit) for bit in format(received_word, f"0{n}b")],
18                dtype=np.uint16)
19            msg, corrected, uncorrectable = decoder_model.decode(
20                received_bits)
21            received_word_array_with_errors.append(received_bits)
22            file.write(" ".join([
23                format(received_word, f"0{n}b"),
```

```

22         "".join(map(str, msg)),
23         format(mask, f"0{n}b") if uncorrectable is False else
24         format(0, f"0{n}b"),
25         str(int(corrected)),
26         str(int(uncorrectable)),
27     ]) + "\n")
28     return np.array(received_word_array_with_errors), msg, error_masks,
29         corrected, uncorrectable, len(error_masks)
30
31 import os
32 import shutil
33 max_errors = 4 # 4 errors limit
34 filename_codeword_with_errors = "outputs/decoder/decoder_top_testing/
35     codeword_with_errors.txt"
36 received_word_array_with_errors, msg, error_masks, corrected, uncorrectable
37     , n_lines = write_codewords_with_error(filename_codeword_with_errors,
38     encoder_output[3838], max_errors)
39 print(f"{n_lines} codewords written to {filename_codeword_with_errors}")
40 # Copia para las secuencias UVM
41 uvm_sequences_dir = "../implement/golay_decoder/decoder/decoder.uvm/sequences"
42 os.makedirs(uvm_sequences_dir, exist_ok=True)
43 uvm_codeword_with_errors = shutil.copy(filename_codeword_with_errors,
44     uvm_sequences_dir)
45 print(f"{n_lines} codewords written to {uvm_codeword_with_errors}")
46 len(received_word_array_with_errors)

```

Además se genera un conjunto de vectores por cada submódulo. Los archivos de los vectores generados se encuentran en "path : /home/abel/Desktop/desktop/github_eros/fec-lab2/model/outputs/decoderyencoder"

2.3. Clase GaloisField en GF(2)

Para comenzar se instanció la clase previamente desarrollada para modelar campos de Galois con parametros $m = 1$ y $P(x) = x + 1$, para testear y corregir posibles issues. La implementación existente no requirió modificaciones y se validaron todas sus operaciones exitosamente:

```

Field Closed Successfully!, 1 Non-Zero Elements
=====
1 -- 0
0 -- -1
=====

ADD:
gf.add(1,1)    = 0
gf.add(1,0)    = 1
gf.add(0,1)    = 1
gf.add(0,0)    = 0

MUL:
gf.mul(1,1)    = 1
gf.mul(1,0)    = 0
gf.mul(0,1)    = 0
gf.mul(0,0)    = 0

INV:
gf.inv(1)      = 1
gf.inv(0)      = Raises: ZeroDivisionError

DIV:
gf.div(1,1)    = 1
gf.div(1,0)    = Raises: ZeroDivisionError
gf.div(0,1)    = 0
gf.div(0,0)    = Raises: ZeroDivisionError

POW:
gf.pow(1,0)    = 1
gf.pow(1,1)    = 1
gf.pow(1,2)    = 1
gf.pow(0,0)    = 1
gf.pow(0,1)    = 0
gf.pow(0,2)    = 0

```

Figura 3: Verificación de las operaciones en GF2.

Una vez verificado el funcionamiento de la clase, se implementaron los metodos para realizar el producto vector-matriz, matriz-matriz, peso de Hamming y algunos metodos auxiliares que se usaron en los modelos del encoder y decoder.

Usando estos nuevos metodos se verificaron las principales propiedades para las matrices del codigo ($GH^T = 0$ y $B^2 = I$):

```

Matrix G (systematic):
=====
[[1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 0 0 0 1 1 1 1]
[0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 1 0 0 1 1 1]
[0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 1 0 1 0 1 0 1 1 1]
[0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 0 1 0]
[0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0 1 1 1 1 0 1 0 0 0 1]
[0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 1 1 0 0]
[0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1 0 0 1 1 1 1 0 1]
[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1 0 1 1 1 1 1 0]
[0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 1 1 1 1 0 1 1]
[0 0 0 0 0 0 0 0 0 1 0 0 1 1 1 0 0 1 1 1 0 1 0 0]
[0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 1 0 0 0 1 1 0 1 0]
[0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 1 0 0 0 1 1 0 1]
[0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 1 0 1 0 1 0 1]]

Matrix H:
=====
[[1 0 0 1 1 1 0 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 1 1 1 0 0 1 1 1 1 0 1 0 0 0 0 0 0 0 0 0]
[0 0 1 1 0 1 0 1 0 1 0 1 1 1 0 0 1 0 0 0 0 0 0 0]
[1 0 1 1 1 1 1 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0]
[1 1 0 1 1 1 1 0 1 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0]
[0 1 1 1 1 1 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[0 1 0 1 0 0 1 1 1 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0]
[0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 0 0 1 0 0 0 0]
[1 0 0 0 0 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 1 0 0 0]
[1 1 1 0 0 1 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0]
[1 1 1 1 0 0 0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0]
[1 1 1 0 1 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1]]

Property: G*HT = 0
=====
[[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0]]

Property: B*B = I
=====
[[1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 1 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 1 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 1 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 1 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 1 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 1 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 1 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 1 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 1]]

```

Figura 4: Matrices y propiedades del código verificadas.

2.4. Codificador y decodificador de referencia

Se implementa el codificador matricial y el decodificador en los archivos `GolayEncoder.py` y `GolayDecoder.py`, respectivamente.

La tabla de síndromes se encuentra en el archivo `fec-lab2/model/outputs/syndromes/error_syndromes.txt`. Estos son generados con los scripts del golden model. Los vectores se presentan de la siguiente forma:

1. Palabra recibida	2. Palabra incorregible
---------------------	-------------------------

Tabla 3: Descripción de las columnas de salida del decodificador.

La totalidad de los errores que se pueden producir dentro de los 24 bits de una palabra recibida en el decoder son generados de la siguiente forma:

El orden en que se encuentran las variables es el siguiente:

1. Palabra recibida	2. Mensaje	3. Máscara de error	4. Palabra corregida	5. Palabra incorregible
---------------------	------------	---------------------	----------------------	-------------------------

Tabla 4: Descripción de las columnas de salida del decodificador.

Los valores decodificados se encuentran en el archivo `../outputs/decoder/decoder_top_testing/codeword_with_errors.txt`.

2.5. Caracterización del decodificador

Para cada peso entre 0 y 4, se recorre la totalidad de los patrones de error correspondientes a dicho peso. Estos vectores son generados en el golden model `golay_decoder.ipynb`.

Luego, estos vectores son procesados por un objeto decodificador para obtener el archivo de vectores `outputs/decoder/decoder_top_testing/codeword_with_errors.txt`.

Este archivo tiene la siguiente convención de vectores.

El script con el cual se genera el conjunto de vectores es el siguiente:

1. Palabra recibida	2. Mensaje	3. Máscara de error	4. Palabra corregida	5. Palabra incorregible
---------------------	------------	---------------------	----------------------	-------------------------

Listing 2: Generación de los vectores de prueba del decodificador

```

1  def write_codewords_with_error(filename, codeword, max_errors):
2      n = len(codeword)
3      received_word_array_with_errors = []
4      # Convertir codeword [0,1,1,...] -> entero
5      codeword_int = int("".join(map(str, codeword)), 2)
6      # Todas las posiciones posibles donde colocar los errores, por peso
7      # C(24,0) + C(24,1) + C(24,2) + C(24,3) + C(24,4) = 12951
8      error_masks = [
9          sum(1 << (n - 1 - pos) for pos in error_positions)
10         for n_errors in range(max_errors + 1)
11         for error_positions in combinations(range(n), n_errors)
12     ]
13     with open(filename, "w") as file:
14         for mask in error_masks:
15             received_word = codeword_int ^ mask
16             received_bits = np.array(
17                 [int(bit) for bit in format(received_word, f"0{n}b")],
18                 dtype=np.uint16)
19             msg, corrected, uncorrectable = decoder_model.decode(
20                 received_bits)
21             received_word_array_with_errors.append(received_bits)
22             file.write(" ".join([
23                 format(received_word, f"0{n}b"),
24                 "".join(map(str, msg)),
25                 format(mask, f"0{n}b") if uncorrectable is False else
26                 format(0, f"0{n}b"),
27                 str(int(corrected)),
28                 str(int(uncorrectable)),
29             ]) + "\n")
30     return np.array(received_word_array_with_errors), msg, error_masks,
31           corrected, uncorrectable, len(error_masks)
32
33 import os
34 import shutil
35 max_errors = 4 # 4 errors limit
36 filename_codeword_with_errors = "outputs/decoder/decoder_top_testing/
37     codeword_with_errors.txt"
38 received_word_array_with_errors, msg, error_masks, corrected, uncorrectable
39 , n_lines = write_codewords_with_error(filename_codeword_with_errors,
40     encoder_output[3838], max_errors)
41 print(f"{n_lines} codewords written to {filename_codeword_with_errors}")
42 # Copia para las secuencias UVM
43 uvm_sequences_dir = "../implem/golay_decoder/decoder/decoder.uvm/sequences"
44 os.makedirs(uvm_sequences_dir, exist_ok=True)
45 uvm_codeword_with_errors = shutil.copy(filename_codeword_with_errors,
46     uvm_sequences_dir)
47 print(f"{n_lines} codewords written to {uvm_codeword_with_errors}")
48 len(received_word_array_with_errors)

```

Además se genera un conjunto de vectores por cada submódulo. Los archivos de los vectores generados se encuentran en "path : /home/abel/Desktop/desktop/github_epos/fec-lab2/model/outputs/decoderyencoder"

3. Parte C — Implementación RTL

3.1. Desarrollo y verificación de módulos comunes

Se comenzo describiendo en Verilog los módulos comunes que serán utilizados posteriormente como parte de los bloques correspondientes al encoder y al decoder.

Una vez implementados, los módulos fueron verificados mediante un testbench sencillo en UVM, con secuencias básicas para validar el funcionamiento de cada uno.

3.2. Analisis: Posible modificacion del decoder

Para el decoder la arquitectura propuesta plantea el cálculo del síndrome y del vector q en etapas diferentes del pipeline. A partir de las propiedades del código, en particular de la condición $B^2 = I$, se analizó una posible modificación de la arquitectura en la cual se calculan el syndrome y q en la primera tapa del pipeline directamente usando el vector recibido r .

Considerando al vector r por sus componentes r_h (alta) y r_l (baja), y como el síndrome:

$$s = r_h \cdot B + r_l$$

Como el vector q se obtiene a partir del síndrome, si aprovechamos la propiedad $B^2 = I$ se puede expresar q directamente en función de los componentes del vector recibido:

$$\begin{aligned} q &= s \cdot B \\ &= (r_h \cdot B + r_l) \cdot B \\ &= r_h \cdot B^2 + r_l \cdot B \\ &= r_h \cdot I + r_l \cdot B \\ &= r_h + r_l \cdot B \end{aligned}$$

De esta manera se permite plantear una alternativa donde el síndrome y el vector q serían calculados a partir de r durante la primera etapa del pipeline.

Sin embargo, el análisis de la implementación mostró que la modificación requiere una cantidad adicional de lógica que no resultaba conveniente para la arquitectura propuesta. En particular, se agregan 12 compuertas XOR adicionales, junto con la lógica necesaria para seleccionar el cálculo correspondiente al síndrome o al vector q .

Considerando tanto el incremento logica como un analisis basico de la actividad de switching de los modulos cuando cambia el vector r, se descartó esta alternativa. Por lo tanto, se mantuvo la arquitectura originalmente propuesta, conservando el cálculo del síndrome y del vector q en las etapas de pipeline inicialmente definidas.

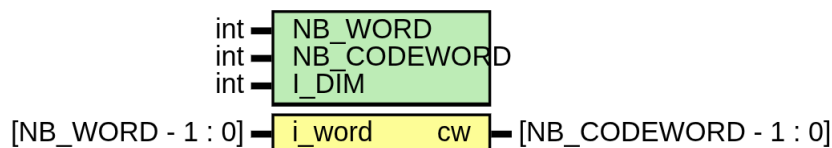
3.3. Implementación RTL del encoder

3.4. Resumen del encoder

Entity: golay_encoder

• File: golay_encoder.sv

Diagram



Generics

Generic name	Type	Value	Description
NB_WORD	int	12	
NB_CODEWORD	int	24	
I_DIM	int	12	

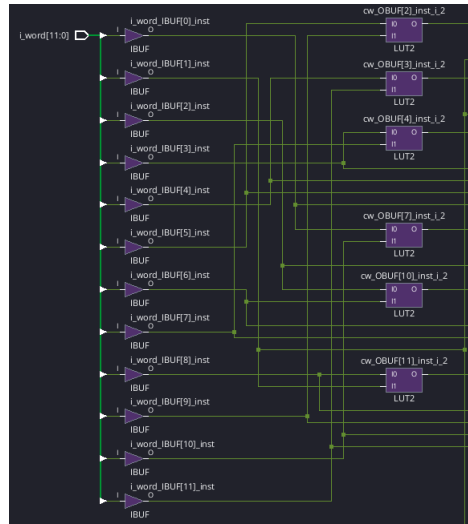
Ports

Port name	Direction	Type	Description
i_word	input	[NB_WORD - 1 : 0]	
cw	output	[NB_CODEWORD - 1 : 0]	

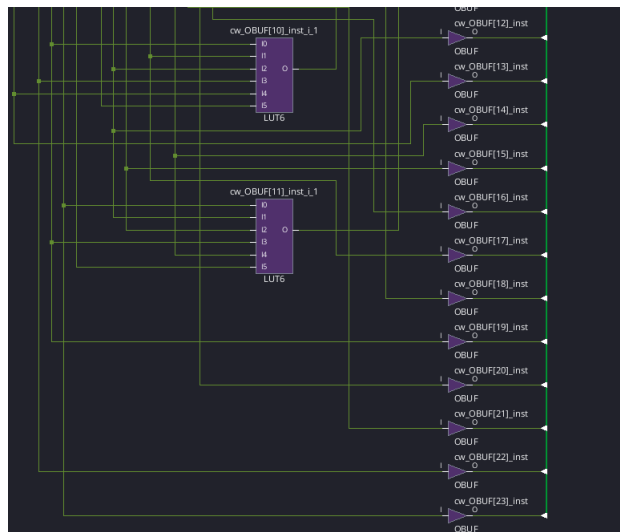
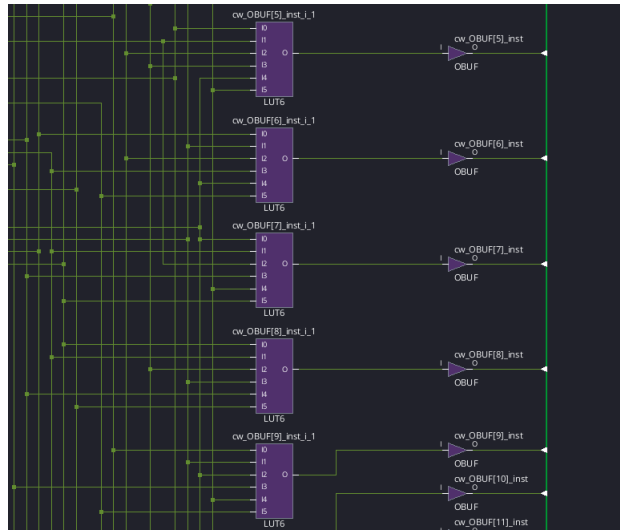
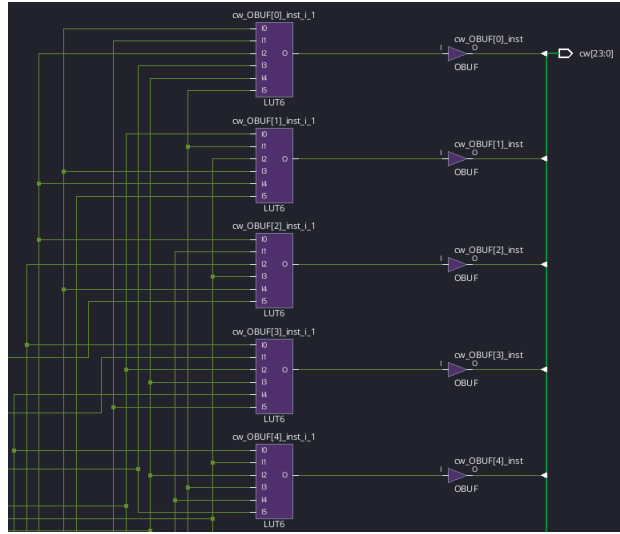
Figura 5: Resumen de la implementación RTL del encoder.

Se diseñó el módulo de codificación de codewords para realizar la multiplicación de la información recibida con la matriz $\mathbf{G} = I|B$.

Se presenta la implementación del codificador. En la siguiente figura se muestra la primer etapa de buffers y las primeras 6 LUTs del circuito.



Luego se presentan las siguientes 12 LUTs con los buffers de salidas del módulo.



3.5. Validación del encoder

Se realiza la simulación con el objetivo de verificar la generación de las codewords a partir de las 4096 combinaciones posibles de la palabra de 24-bits a codificar. Las posibles combinaciones y las codewords resultantes se obtienen del golden model realizado en software. El formato del archivo es el siguiente:

Columna 1	Columna 2
Información	Codeword

Se muestra la simulación realizada en Vivado:

Name	Value												
		0.000 ns	20.000 ns	40.000 ns	60.000 ns	80.000 ns	100.000 ns						
> i_word[11:0]	000	000	001	002	003	004	005	006	007	008	009	00a	00b
> cw[23:0]	000000	000000	001ea9	002f1a	0031b3	004e74	0050dd	00616e	007fc7	00887b	0096d2	00a761	0...
> exp_cw[23:0]	000000	000000	001ea9	002f1a	0031b3	004e74	0050dd	00616e	007fc7	00887b	0096d2	00a761	0...
> line_num[31:0]	00000001	00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008	00000009	0000000a	0000000b	0...
> n_vectors[31:0]	00000000	00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008	00000009	0000000a	0000000b	...

Figura 6: Verificación del encoder

Mediante la consola de TCL podemos ver que la implementación en RTL procesa la información correctamente.

```

# Tcl console output
launch_simulation: Time (s): cpu = 00:00:11 ; elapsed = 00:00:10 .
run -all
-----
golay_encoder: 4096 vectors, 0 failed
RESULT: PASS
-----
$finish called at time : 40960 ns : File "/home/abel/Desktop/desktop

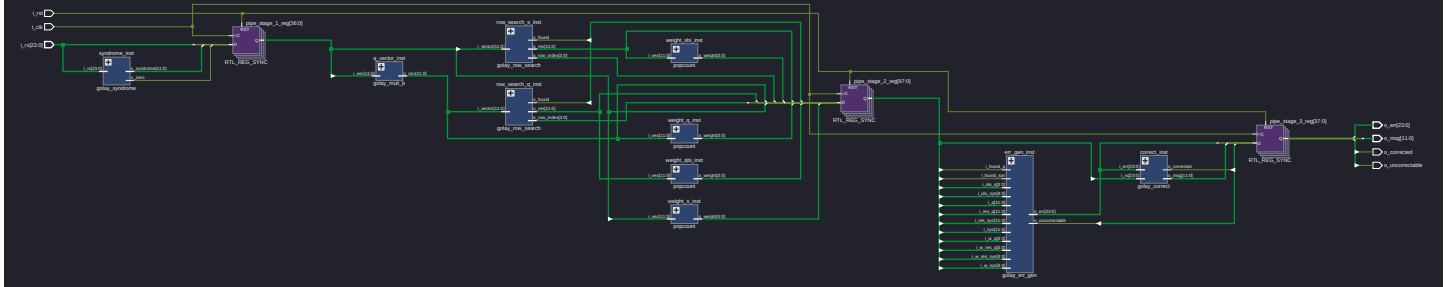
```

3.6. Implementación RTL del decoder

A partir de los módulos comunes, se describieron bloques para el cálculo del síndrome, del vector q , la búsqueda por filas y la generación de la máscara de error mediante los diferentes casos en la decodificación.

Cada uno fue validado mediante el testbench UVM, utilizando secuencias de estímulos básicas para verificar su funcionamiento.

Una vez validados los módulos, se incorporaron los registros de pipeline correspondientes para integrar el top del decoder. Quedando la arquitectura final y cada etapa lógica:



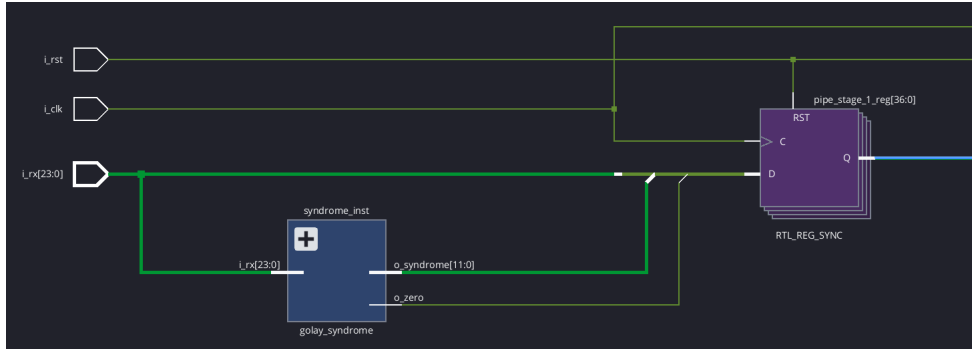


Figura 7: Primera etapa del pipeline del decoder.

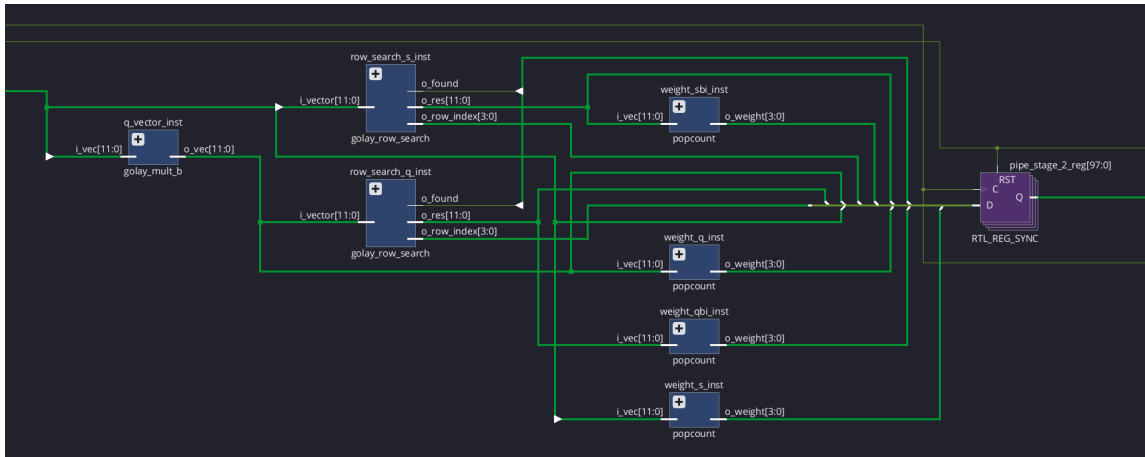


Figura 8: Segunda etapa del pipeline del decoder.

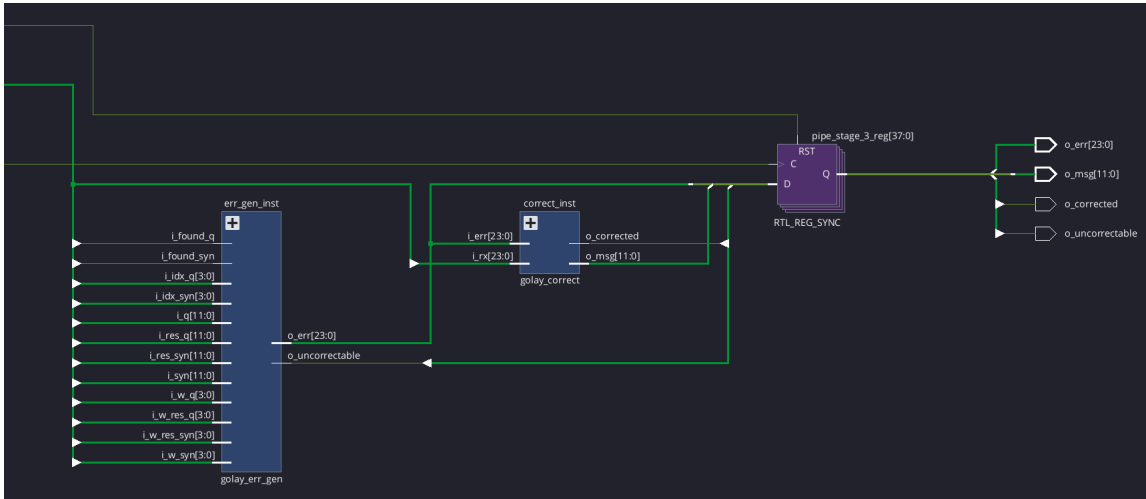


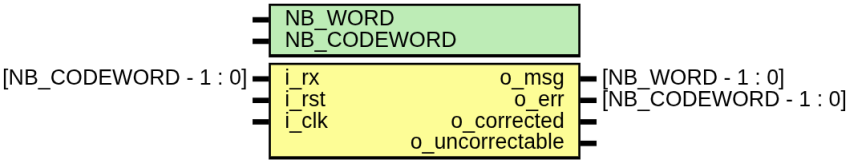
Figura 9: Tercera etapa del pipeline del decoder.

3.7. Resumen del decoder

Entity: decoder

• File: decoder.sv

Diagram



Generics

Generic name	Type	Value	Description
NB_WORD		12	
NB_CODEWORD		24	

Ports

Port name	Direction	Type	Description
o_msg	output	[NB_WORD - 1 : 0]	
o_err	output	[NB_CODEWORD - 1 : 0]	
o_corrected	output		
o_uncorrectable	output		
i_rx	input	[NB_CODEWORD - 1 : 0]	
i_rst	input		
i_clk	input		

Figura 10: Resumen de la implementación RTL del decoder.

3.8. Validacion del decoder

Se realizó su verificación funcional usando el entorno UVM mediante vector matching, con archivos de referencia generados a partir del modelo desarrollado previamente. En los vectores cada línea contiene el vector de entrada *rx* y las salidas esperadas correspondientes al mensaje decodificado, el patrón de error y las flags corrected y uncorrectable.

Como primera verificación funcional, se utilizaron codewords sin errores. En este caso se verificó que el patrón de error sea nulo, y que las señales corrected y uncorrectable permanecieran en bajo durante todo el test. La Figura 11 muestra los resultados obtenidos durante el test:

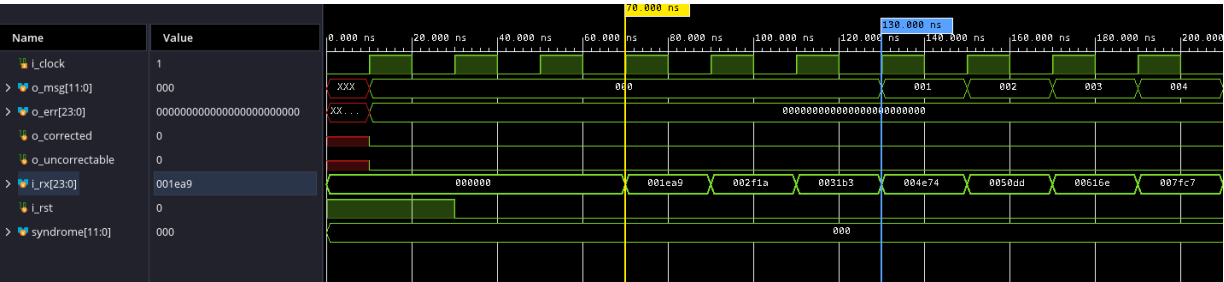


Figura 11: Verificación del decoder utilizando codewords sin errores.

En esta ventana se observa también la latencia de tres ciclos esperada. Sobre el primer cursor, en

hexadecimal los tres dígitos superiores de rx corresponden al mensaje 001h junto con los bits de paridad correspondientes EA9h. Luego de tres ciclos, se obtiene a la salida el mismo mensaje decodificado.

3.9. Correcciones en el decoder

A continuación, se realizó una segunda prueba aplicando en cada codeword todos los patrones de error posibles correspondientes a pesos de error entre cero y cuatro.

Durante la verificación se detectaron algunos errores en el decoder. Los problemas estuvieron relacionados a condiciones en la generación de la máscara de error y al conexionado entre los diferentes módulos.

Por observación y análisis, se identificaron los problemas y se realizaron las correcciones correspondientes. Luego de aplicar las correcciones, se logró validar el funcionamiento del decoder para los casos contemplados en el test.

Se puede ver un waveform general del test donde, muy resumidamente, luego de salir del reset se aplican errores de peso 0 a 3 donde se recupera el mensaje original (EFEh) y se reportan los casos como corrected. Luego se insertan errores de peso 4 hasta el final del test y se reportan como uncorrectable:



Figura 12: Simulación del decoder para los patrones de error evaluados.

Y también se agregó un objeto en el testbench para reportar la cantidad de patrones de error por peso (en formato de array `error_weight[weight]`) y la cantidad de casos corregidos y detectados. Se obtuvieron los valores de la tabla verificada en la PARTE B (Ejercicio 7 Caracterización del decodificador):

Peso de e	0	1	2	3	4
Patrones	1	24	276	2024	10626
Corregidos	1	24	276	2024	0
Detectados	0	0	0	0	10626

Figura 13: Tabla de casos de decodificación según peso de error (modelo).

```

UVM_INFO /home/clifferto/Documents/fec-lab2/implen/golay_decoder/decoder/decoder.uvm/tb_test.svh(29) @ 0: uvm_test_top [TEST] Resetting DUT
UVM_INFO /home/clifferto/Documents/fec-lab2/implen/golay_decoder/decoder/decoder.uvm/tb_virtual_seq.svh(14) @ 30000: reporter@vseq [vseq]
Launching codewords_with_error_seq...

relaunch_sim: Time (s): cpu = 00:04:12 ; elapsed = 00:02:38 . Memory (MB): peak = 11417.199 ; gain = 0.000 ; free physical = 550 ; free virtual = 20019
run all
UVM_INFO /home/clifferto/tools/2025.2/Vivado/data/system_verilog/uvm_1.2/xlnx_uvm_package.sv(19968) @ 259130000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
-----
Name          Type          Size  Value
-----
report         tb_report      -    @575
error_weight   sa(integral)   5     -
[0]            integral      32    'd1
[1]            integral      32    'd24
[2]            integral      32    'd276
[3]            integral      32    'd2024
[4]            integral      32    'd10626
corrected      sa(integral)   5     -
[0]            integral      32    'd0
[1]            integral      32    'd24
[2]            integral      32    'd276
[3]            integral      32    'd2024
[4]            integral      32    'd0
detected       sa(integral)   5     -
[0]            integral      32    'd0
[1]            integral      32    'd0
[2]            integral      32    'd0
[3]            integral      32    'd0
[4]            integral      32    'd10626
-----
UVM_INFO /home/clifferto/tools/2025.2/Vivado/data/system_verilog/uvm_1.2/xlnx_uvm_package.sv(13673) @ 259130000: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 6
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
[RNTEST] 1
[TEST] 1
[TEST_DONE] 1
[UVM/COMP/NAMECHECK] 1
[UVM/RELNOTES] 1
[vseq] 1

```

Figura 14: Tabla de casos de decodificación según peso de error (RTL).

Finalmente, se verificó el funcionamiento del decoder utilizando las codewords correspondientes a la PARTE A (Ejercicio 3 Decodificación manual). Las tres codewords fueron aplicadas como vectores de entrada y se verificó su correspondiente decodificación. La Figura 15 muestra las señales obtenidas durante esta prueba.

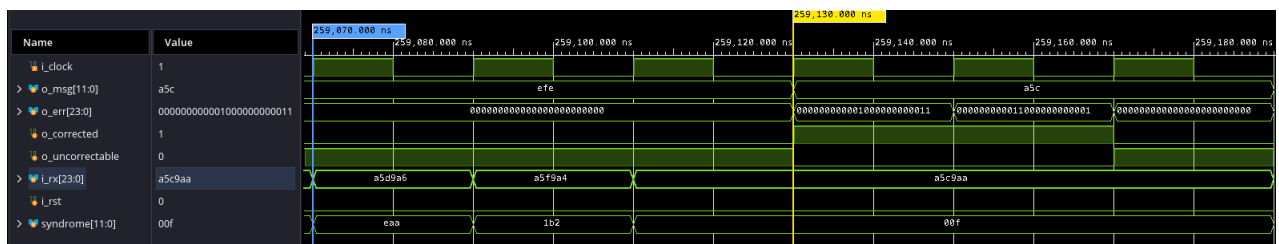
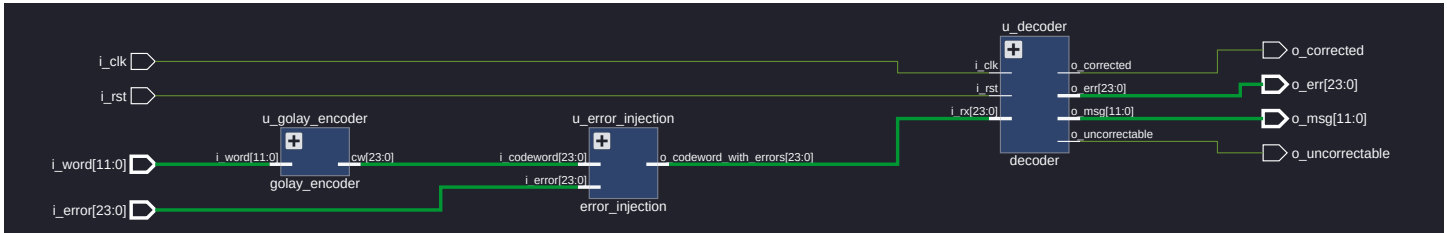


Figura 15: Decodificación de las tres codewords correspondientes a la PARTE A.

3.10. Integración del top level

En esta etapa se realizó la integración de los módulos desarrollados previamente obteniendo el top level del diseño. Para ello, se instanciaron los módulos encoder, decoder e inyector de errores, estableciendo las conexiones necesarias entre ellos de acuerdo con la arquitectura definida.



3.11. Verificación del top level

Una vez integrado el top level, se generaron los vectores necesarios para validar el funcionamiento de la arquitectura final. Cada línea del archivo de vectores contiene, en orden, la palabra a transmitir y el patrón de error a aplicar. Como resultados esperados se incluyen el mensaje decodificado, la máscara de error detectada y las señales corrected y uncorrectable.

La validación se realizó mediante el entorno UVM, utilizando vector matching entre los resultados obtenidos por el diseño RTL y los valores generados previamente a partir del modelo de referencia. La Figura 16 muestra una vista general de la simulación correspondiente al test, donde se aplican patrones de error de peso menor o igual a tres, seguida por una región en la que se aplican patrones de error incorregibles.

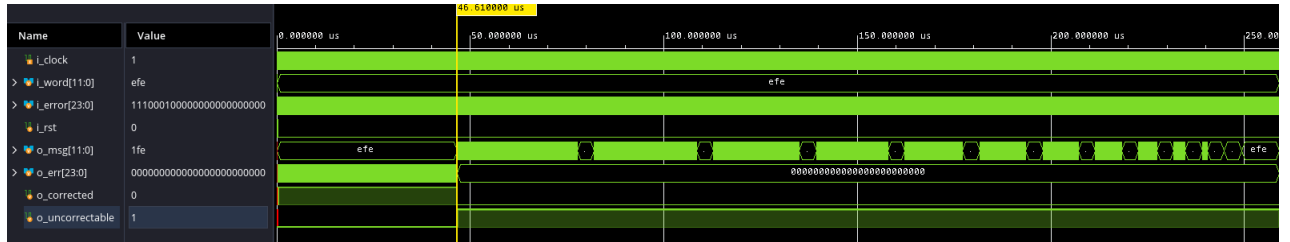


Figura 16: Waveform general correspondiente a la verificación del top level.

Al salir del reset se aplica la palabra EFEh sin ningún patrón de error, luego de tres ciclos se obtiene el mensaje decodificado y el patrón de error detectado es nulo. Este comportamiento se muestra en la Figura 17.

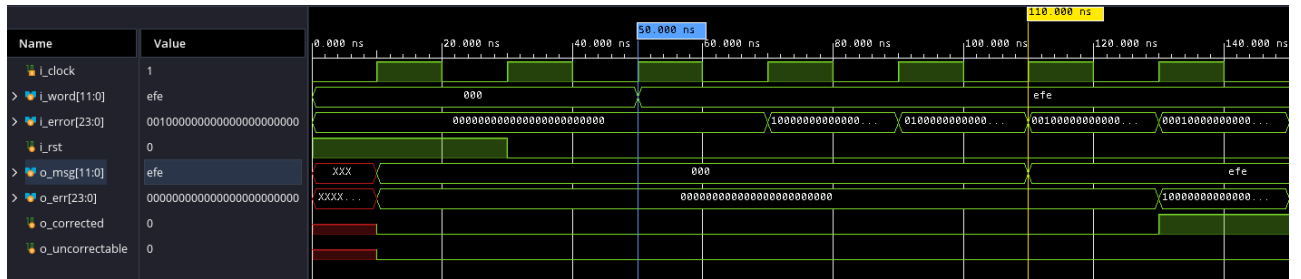


Figura 17: Decodificación sin errores y verificación de la latencia.

A continuación, se incrementó el peso del patrón de error aplicado. En la Figura 18 se muestra la transición desde un patrón de error de peso uno hacia uno de peso dos.

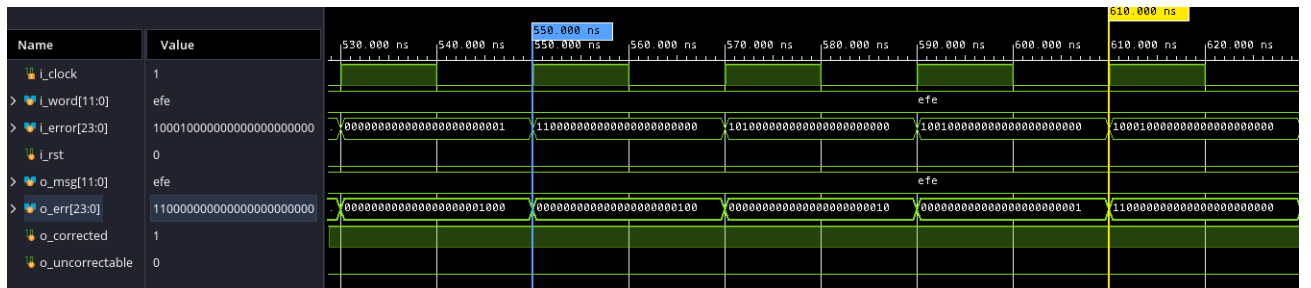


Figura 18: Transición entre patrones de error de peso uno y dos.

La Figura 19 muestra la transición desde un patrón de error de peso dos hacia uno de peso tres.

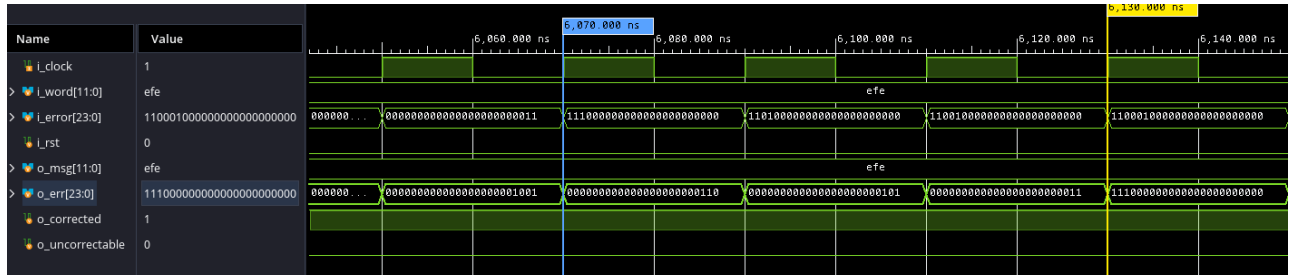


Figura 19: Transición entre patrones de error de peso dos y tres.

Finalmente, en la Figura 20 se observa la transición hacia un patrón de error de peso cuatro. A partir de este punto, la señal uncorrectable permanece en alto, indicando que el patrón de error aplicado no puede ser corregido por el decoder. En esta condición, las salidas correspondientes al mensaje y al patrón de error detectado se consideran inválidas.

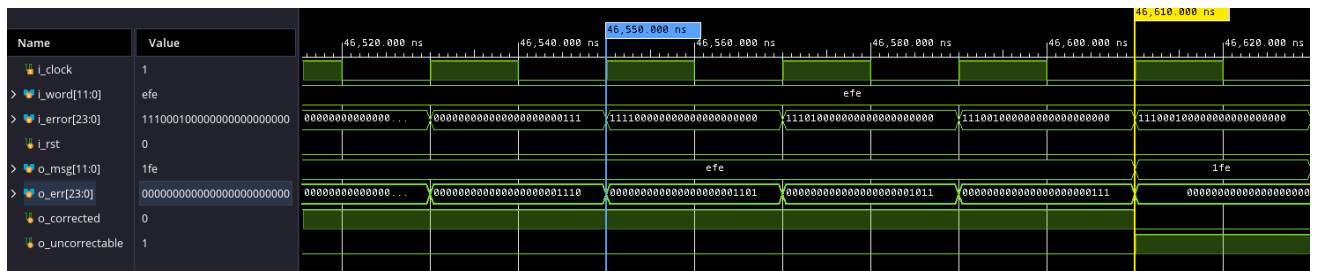


Figura 20: Transición entre patrones de error corregibles e incorregibles.